NAME: NGO TUNG CHUONG

ID: S225586848

1. Study the Introspective Sort, which is a default sorting algorithm implemented in the Array.Sort method. You must address the following questions:
   - What is special about the Introspective Sort?
   - Does this method result in an unstable sorting; that is, if two elements are seen equal, might their order be not preserved after the data collection is sorted? Or does it perform a stable sorting, which preserves the order of elements that are seen equal?
   - What is the time complexity of this sorting algorithm?

The special about the Introspective Sort is that it is a HYBIRD ALGORITHM. By default, it started with Quick sort, which is high-speed efficiency. And Insertion sort to finish off any short and tiny chucks elements (usually below 20). Also, it keeps Heap sorts on standby as a safety net to guarantee that the sorting never be stuck in an infinity loop.

Quick answer is NO; this method would likely result in unstable sorting. Due to fact that it is a "HYBIRD" built from components that are themselves unstable, it inherits that instability.

The time complexity of Introspective Sort capped with $0(n * logn)$ in all cases: best, average and worst. Even though, it used Quick Sort and Insertion Sort with worst case is $0(n^2)$. But Intro sort monitors the recursive depth. So that when it is noted that the depth has exceeded a limit (usually $2logn$), it automatically forces a switch to Heap sort. About Inserting sort, although it did contribute to the process. But only very small subarrays are applied (fewer than 20 elements). So, it does not affect much in the process overall.

2. In Task 1.1P on the Vector<T> class, you have completed/been provided with a number of methods (and properties), such as Count, Capacity, Add, IndexOf, Insert, Clear, Contains, Remove, and RemoveAt. Answer the following questions:
   - What is the time complexity of each of these operations?
   - Does your implementation match the complexity of the equivalent operations provided by the Microsoft .NET Framework for its List<T> collection?

TIME COMPLEXITY

MEMORY ACCESS

- Count: $O(1)$
- Capacity: $0(1)$

PROCESS

- Add: $O(1) - Worst\ Case(Array\ full): O(n)$
- IndexOf: $O(n)$
- Insert: $O(n)$
- Clear: $O(n)$
- Contains: $O(n)$
- Remove: $O(n)$
- RemoveAt: $O(n)$

Yes, in terms of Big-O classification, my implementation matches the complexity of the equivalent operations provided by Microsoft .NET Framework for its List<T> collection.

3. Answer and **explain** whether the following statements are right or wrong.

3.a. A $\theta(2^n)$ algorithm **always** takes longer to run than a $\theta(\log n)$ algorithm.

3.b. The best-case time complexity of the Bubble Sort algorithm is always $O(n)$.

3.c. The worst-case time complexity of the Insertion Sort algorithm is always $O(n^2)$.

3.d. The Selection Sort is an in-place sorting algorithm.

3.e. The worst-case space complexity of the Insertion Sort algorithm is $O(n)$.

3.f. $1 + 10 = o(1)$

3.g. $n^3 + 10^6 n = O(n)$

3.h. $\mathrm{n} \log n = O(\log n)$

3.i. $100 \log n = O(n)$

3.j. $\log n + 2^{100} = \Theta(\log n)$

3.k. $n \log n = \Omega(\log n)$

3.l. $\mathrm{n} + 2^n = \Theta(n)$

3.m. $n + \frac{100n}{\log n} = o(\log n)$

3.n. $\mathrm{n}! = O(n)$

3a. **Correct**. Basic mathematic, $2^n always\ larger\ (logn)$.
(**Wrong**. In most case the 2^n totally "out slow" the time to run than a $\log(n)$ algorithm. But in few small cases, where n is 0 or 1. The "setup" time for the log(n) algorithm made it take longer than expect in real-life scenario. Like 10 + log(n), so with n = 1. The time complexity would be 10 + log(1) ≈11. Meanwhile, with 2^n the step would like 1,2,3,4,5 and done.

3b. **Correct**. In best case scenario, if the input array is already sorted, Bubble sort will only perform one pass through the array to confirm the order, making it O(n).
3c. **Correct**. Insertion sort is the algorithm that works by shifting the small element on the front. So, in worst case, where the input array is reverse sorted. It would take $O(n^2)$.
3d. **Correct**. Selection Sort only takes one extra memory to do the swapping, which is not large enough to not be considered an in-place sorting algorithm.
3e. **Wrong**. The worst case of Insertion Sort is always $O(n^2)$.
3f. **Correct**. 1 + 10 equals constant 11. Constant-time operations are classified as O(1).
(**Wrong**. In contrast with **Big-O**, indicate that the **Upper Bound** ($f(n) \leq g(n)$). The Little-o, show up with a stricter rule, **Strict Upper Bound** ($f(x) < g(n)$). In this case, to make the statement stays TRUE. The 1 + 10 have to be equals 0, 0 = o(1). But 1 + 10 cannot be 0, so the statement is incorrect.)

3g. **Wrong**. The expression is grow faster than n.

3h. **Wrong**. Because the symbol is upper bound. The upper bound of n*log(n) is n*log(n).
3i. **Correct**. 100*log(n) grow no faster than n.
3j. **Correct**. Ignore constant 2^{100} then it corrects.
3k. **Correct**. Because the symbol is lower bound. The lower bound of n*log(n) is log(n).
3l. **Wrong**. The term n + 2^n grow faster than n.
3m. **Wrong**. To be the upper bound a function must be at least as fast as it expression.
3n. **Wrong**. n! grow significantly faster than n.

Notation	Meaning	Analogy
O **(Big O)**	Upper Bound	"It won't take longer than..."
Ω **(Big Omega)**	Lower Bound	"It will take at least..."
Θ **(Big Theta)**	Tight Bound	"It will take exactly..."

4. Give your best- and worst-case asymptotic runtime analysis to the following code snippets.

4.1. Let **flag** be a random Boolean variable, whose value is either **true** or **false.**

```
int count = 0;
for (int i = 0; i < n; i++)
{
   if (flag)
   {
       for (j = i+1; i < n; j++)
       {
           count = count + i + j;
       }
   }
}
```

Best case scenario, flag = false. The runtime would be O(n).
Worst case scenario, due to the infinity condition: **"i < n"** the runtime complexity would be $\boldsymbol{O}(\infty)$**.**

4.2. Let random() be a function producing a real random number in the range [0,1).

```
int count = 0;
for (int i = 0; i < n; i++)
{
   int num = random();
   if( num < 0.01 )
   {
       count = count + 1;
   }
}
int num = count;
for (int j = 0; j < num; j++)
{
   count = count + j;
}
```

Best case scenario, the runtime would be O(n).
Worst case scenario, the runtime would be $O(n)$.

4.3. Let Compare(x, y) be a method with the time complexity of $\Theta(1)$.

```
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n-i-1; j++)
   {
       if (a[j] > a[j+1])
       {
           Compare(a[j], a[j + 1]);
       }
   }
}
```

Best case scenario, the runtime is $O(n^2)$
Worst case scenario, the runtime is $O(n^2)$